

Classificatori Lineari

Nearest Neighbor Classifier (NNC)

L'esempio più semplice di classificatore è il **Nearest Neighbor Classifier (NNC)**.

Funzionamento

1. L'NNC possiede in memoria il **training set completo**
2. Per classificare un'immagine, computa la **distanza** tra l'immagine e tutte le immagini del training set
3. Individua l'immagine a **distanza minima**
4. Classifica l'immagine con la stessa label dell'immagine più vicina

Metriche di Distanza

La distanza tra due immagini può essere definita con varie metriche: - **Errore quadratico medio (MSE)** - **Mutua informazione** - **Distanza L1 (Manhattan)** - **Distanza L2 (Euclidea)**

Suggerimento: Registrazione Visto che la posizione di un oggetto può essere diversa, è ragionevole far precedere il computo della distanza dalla **registrazione** delle due immagini, oppure il training set deve contenere tutte le possibili trasformazioni geometriche.

Limiti dell'NNC

Problema	Descrizione
Prestazioni basse	Difficoltà di individuare una metrica efficiente
Complessità computazionale	Per ogni classificazione, confronto con TUTTO il training set
Memoria	Necessità di mantenere l'intero training set in memoria

k-Nearest Neighbor Classifier (k-NN)

Il **k-Nearest Neighbor Classifier (k-NNC)** estende l'NNC considerando le **k immagini** con distanza minore invece dell'unica immagine più vicina.

Algoritmo

1. Trova le **k immagini** più vicine nel training set
2. Implementa una **“votazione”**: ogni immagine vota la propria label
3. Sceglie la label che ottiene la **maggioranza relativa**

Info: Caso Particolare Per **k=1**, il k-NNC corrisponde all'NNC.

Scelta di k

- **k piccolo**: più sensibile al rumore
- **k grande**: confini di decisione più lisci, ma può includere classi diverse

La complessità computazionale rimane simile all'NNC se $k \ll$ dimensioni del training set.

Classificatore Lineare

L'approccio k-NNC presenta due problemi fondamentali: 1. Necessità di **mantenere in memoria tutto il training set** 2. Alta **complessità computazionale**

Per questi motivi, si preferisce introdurre una **funzione score** e una **funzione loss**.

Score Function

Sia: - D = dimensione dell'immagine (numero di pixel) - K = numero di classi

La funzione score è:

$$f : \mathbb{R}^D \rightarrow \mathbb{R}^K$$

$$f(I, W, b) = WI + b$$

Dove: - W = matrice dei **pesi (weights)** di dimensione $K \times D$ - b = **bias vector** di dimensione K - Ogni riga di W rappresenta un **classificatore** associato a una classe

Suggerimento: Vantaggio Una volta definiti gli iperparametri, la classificazione richiede solo una **singola moltiplicazione di matrici** più un'addizione $\rightarrow$ molto veloce!

Formulazione Alternativa

Si può includere il bias nella matrice W aggiungendo un pixel fittizio di valore 1:

$$f(I, W) = WI$$

Interpretazione Geometrica

- Le immagini sono **punti** in uno spazio D-dimensionale
 - Gli iperparametri W e b rappresentano **K iperpiani** che dividono lo spazio
 - Cambiando gli iperparametri, i piani si spostano e cambia la classificazione
-

Support Vector Machine (SVM)

L'approccio **SVM** definisce una loss function specifica.

Hinge Loss

Per l'immagine i -esima con label vera y_i :

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

Dove: - s_j = score per la classe j - Δ = margine desiderato tra le classi

Esempio: Esempio Numerico Se abbiamo 3 classi e $s = [13, -7, 11]$ con label vera = classe 1 e $\Delta = 10$:

$$\begin{aligned} L_i &= \max(0, -7 - 13 + 10) + \max(0, 11 - 13 + 10) \\ L_i &= \max(0, -10) + \max(0, 8) = 0 + 8 = 8 \end{aligned}$$

Interpretazione

- La loss è **zero** se lo score corretto supera tutti gli altri di almeno Δ
- È **positiva** (penalità) altrimenti
- La funzione $\max(0, -)$ è detta **hinge loss**
- Variante: $\max(0, -)^2$ è detta **squared hinge loss**

Regolarizzazione

Per evitare soluzioni multiple (se W minimizza la loss, anche kW con $k > 1$ lo fa):

$$L = \frac{1}{N} \sum_i L_i + \lambda \sum_k \sum_l W_{k,l}^2$$

Il termine $\lambda \|W\|^2$ (norma L2) privilegia pesi più piccoli e distribuiti.

Info: MATLAB In MATLAB l'approccio SVM è implementato dall'oggetto `ClassificationECOC`.

Softmax Classifier

Un diverso tipo di classificatore lineare usa la **cross-entropy loss**.

Cross-Entropy Loss

Per l'immagine i -esima:

$$L_i = -\log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right)$$

Che si può riscrivere come:

$$L_i = -f_{y_i} + \log \sum_j e^{f_j}$$

Softmax Function

La funzione:

$$\text{softmax}(f)_j = \frac{e^{f_j}}{\sum_k e^{f_k}}$$

mappa valori reali in: - Range $[0, 1]$ - Somma = 1 (interpretabile come **probabilità**)

Interpretazione Probabilistica

Il classificatore Softmax minimizza la **cross-entropy** tra: - Probabilità predette q (output softmax)
- Distribuzione “vera” p (one-hot: tutto 0 tranne la classe corretta che è 1)

$$H(p, q) = - \sum_j p(j) \log q(j)$$

Info: Demo Prova interattiva: <http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>

Gradient Descent

L'addestramento del classificatore è un problema di **minimizzazione della loss** attraverso l'ottimizzazione degli iperparametri.

Idea Base (1D)

Per minimizzare $y = f(x)$ partendo da x_0 :

1. Calcola la derivata $\frac{dy}{dx}$ nel punto corrente
2. Muoviti nella direzione **opposta** al gradiente
3. Ripeti fino alla convergenza

```
x(1) = x0; % condizioni iniziali
for i = 2:N
    if (dy/dx(x(i-1))) < 0
        x(i) = x(i-1) + Dx;
    else
        x(i) = x(i-1) - Dx;
    end
end
```

Derivata Numerica (Differenze Finite)

$$\frac{dy}{dx} \approx \frac{y(x + \Delta x) - y(x - \Delta x)}{2\Delta x}$$

Gradient Descent Multidimensionale

Per problemi con molte variabili, la derivata diventa il **gradiente** (vettore delle derivate parziali):

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]$$

Varianti del Gradient Descent

Variante	Descrizione
Batch GD	Usa tutto il training set per ogni step
Mini-batch GD (MGD)	Usa un sottoinsieme del training set
Stochastic GD (SGD)	Usa una singola immagine per step

Suggerimento: Pratica Nella pratica si usa spesso il termine “SGD” anche per indicare l’approccio mini-batch.

Esempio MATLAB: Classificatore Softmax

```
% Creare l'ImageDatastore
digitDatasetPath = fullfile('TEST1');
testData = imageDatastore(digitDatasetPath, ...
    'IncludeSubfolders', true, 'LabelSource', 'foldernames');

% Dividere in training e test set
CountLabel = testData.countEachLabel;
N = table2array(CountLabel(1,2));
trainingNumImages = 3*N/4;
[trainImageData, testImageData] = ...
    splitEachLabel(testData, trainingNumImages, 'randomize');

% Preparare i dati (X = pixels, T = labels one-hot)
% ... preparazione matrici X e T ...

% Addestrare il classificatore softmax
SoftMaxClass = trainSoftmaxLayer(X, T);

% Valutare sul training set
Y = SoftMaxClass(X);
plotconfusion(T, Y);
```

Confusion Matrix

La **confusion matrix** misura le prestazioni del classificatore: - **Diagonale**: classificazioni corrette (VP, VN) - **Fuori diagonale**: errori (FP, FN)

!Pasted image 20251201191724.png